

Gestão Global de Exceções no NestJS

Arquitetura de erros personalizados e filtros globais para APIs resilientes.

1. A Base: `AppError` (Custom Exception)

A classe `AppError` funciona como a base para todas as exceções controladas da aplicação. Ao estender o `HttpException` do NestJS, garantimos que o erro possa carregar não apenas uma mensagem, mas um estado semântico completo.

O que ele faz?

Encapsula a mensagem, o código HTTP (ex: 404, 400) e metadados extras (como `meta` ou `context`). Isso permite anexar detalhes técnicos que serão processados pelo filtro global.

2. O Sentinela: `HttpExceptionFilter`

Este filtro atua como um interceptor que captura qualquer exceção lançada na aplicação antes dela chegar ao cliente final.

Lógica de Instância: Ele verifica se a exceção é um `AppError`, um `HttpException` padrão ou um erro desconhecido (500), adaptando a resposta JSON de acordo.

- **Padronização:** Garante que o cliente receba sempre o mesmo formato de resposta.
- **Segurança:** O `stack trace` só é exposto se o status for `>= 500`, evitando vazar detalhes da infraestrutura.

3. Registro Global no AppModule

Ao utilizar `{ provide: APP_FILTER, useClass: HttpExceptionFilter }` no `AppModule`, você ativa o comportamento para toda a aplicação.

```
providers: [  
  {  
    provide: APP_FILTER,  
    useClass: HttpExceptionFilter,  
  },  
]
```

Isso elimina a necessidade de adicionar `@UseFilters()` manualmente em cada controller, garantindo que novos módulos já nasçam com esse tratamento de erro.